

Integración Active-IA · AI-Native

Estado de implementación y pedido de definiciones

De: equipo de Active-IA (api.active-ia.com)

Para: equipo de AI-Native (tutor.active-ia.com)

Fecha: 20 de agosto de 2026

Referencia: su documento «Active-IA — cambios necesarios para integrar la corrección con AI-Native» (18/08, actualizado 19/08)

Leímos el documento completo y verificamos cada afirmación contra nuestro código, no contra nuestro recuerdo de él. Descompusimos el pedido en ocho unidades de trabajo y ya hay cuatro en marcha. Este documento tiene dos partes: **qué está construido** y **qué necesitamos de ustedes para seguir**.

La segunda parte es la que importa. Hay siete definiciones pendientes y cinco no dependen de ninguna línea de código.

1. Lo que necesitamos de ustedes

Ordenado por lo que bloquea, no por lo que nos resulta cómodo pedir. Los tres primeros condicionan el piloto.

#	Qué necesitamos	Bloquea	Urgencia
1	La respuesta sobre personería, por escrito. Nos confirmaron verbalmente que podemos continuar y lo agradecemos. Necesitamos el respaldo escrito: el día que un alumno pregunte, «nos dijeron que sí» no alcanza.	Despliegue con datos de alumnos reales	Alta
2	Marcar qué criterios dependen de la ejecución al publicar cada rúbrica (campo nuevo, ver §3).	La garantía que ustedes pidieron sobre <i>compila: false</i>	Alta
3	Confirmar el contrato de escritura contra su doble HTTP antes de que lo desplaguemos.	Deploy de los endpoints de escritura	Alta
4	Por qué canal quieren recibir la credencial de la cuenta de servicio. No la vamos a mandar por mail ni por chat.	Entrega de la cuenta de servicio	Media
5	Una ventana para apuntar su cliente a staging antes de producción.	Primera prueba end-to-end	Media
6	¿Quién puede pedir una anonimización? ¿Solo un administrador nuestro, o también su cuenta de servicio desde su procedimiento automatizado?	Diseño del borrado por alumno	Baja
7	Política de retención de respaldos. Ver la advertencia del §5.	Alcance real del compromiso de anonimización	Baja

2. Qué está construido

Unidad de trabajo	Corresponde a	Estado
Detalle de entrega (500)	su §7.2	Terminado
Nivel de ejercicio + referencia externa	su §3.1 y §3.2	Terminado

Endpoints de escritura	su §3.3	Terminado
Penalizaciones y desglose que no cierran	sus bugs 2 y 3	Análisis cerrado, pendiente de revisión interna
Falsos positivos del motor	sus bugs 1, 4, 5, 6	Especificado, sin empezar
Corrección con resultado de tests	su §3.4	Especificado, sin empezar
Cuenta de servicio	su §3.5	Especificado, sin empezar
Borrado por alumno	su §3.6	Especificado, sin empezar

2.1 Los tres endpoints ya existen

```
POST /api/v1/trabajos-practicos/
PUT /api/v1/trabajos-practicos/by-ref/{external_ref}
GET /api/v1/trabajos-practicos/by-ref/{external_ref}
```

El **PUT por referencia externa es idempotente**, como pidieron: reenviar el mismo cuerpo produce el mismo estado final. Responde **201** cuando creó y **200** cuando actualizó, para que sepan qué pasó sin consultar antes.

La respuesta devuelve, por cada ejercicio, su *external_ref* y su *rubrica_id*. Y agregamos una garantía que no pidieron pero que necesitan:

El rubrica_id de un ejercicio es estable de por vida. Los ejercicios se emparejan por *external_ref*, nunca por orden ni por título. Reordenarlos o renombrarlos en su plataforma no rota ninguna asociación. Si el *rubrica_id* cambiara entre publicaciones, sus correcciones anteriores quedarían colgando de una rúbrica que ustedes ya no vinculan a ese ejercicio.

2.2 Dos hallazgos sobre los bugs que reportaron

El bug 2 no es que el modelo desobedezca

Nuestro backend **nunca aplicó las penalizaciones**. La instrucción vivía únicamente en el texto del prompt, y el cálculo de la nota las ignoraba por diseño — está escrito así en el código, no es un descuido reciente. Es un arreglo determinístico, no de prompt. Lo mismo vale para el criterio que no cierra con sus subcriterios (bug 3).

El bug 1 tiene una causa de una línea

La lista de archivos de la entrega existe en nuestra base y **nunca llegaba al modelo**. Recibía un texto concatenado y tenía que inferir qué había entregado el alumno. Cuando además venía truncado por límite de contexto, veía menos de lo que había.

3. Lo que cambia de su lado

Cuatro cosas. La primera requiere trabajo suyo; las otras tres son para que no los sorprendan.

3.1 Un campo nuevo por criterio: si depende de la ejecución

Nos pidieron que con *compila: false* no cerremos criterios del tipo «el programa funciona». De acuerdo. Pero ponerlo solo en el prompt sería repetir exactamente el error del bug 2: una regla declarada que el motor puede no honrar.

Lo vamos a hacer **determinístico en el backend**. Para eso, la rúbrica tiene que decir qué criterios dependen de que el programa corra. Es un booleano por criterio, opcional, que por defecto va en falso.

Lo que les pedimos: márkennlo al publicar las rúbricas. Si no viene, el comportamiento es el de hoy y la garantía no aplica — no porque no queramos darla, sino porque no tendríamos con qué distinguir un criterio de diseño de uno de funcionamiento.

3.2 Los casos ocultos mal formados los vamos a rechazar, no a limpiar

Compartimos el razonamiento de su §3.3.1: lo que no recibimos no lo podemos citar. Si por un error de su lado llega un caso oculto con su salida esperada o su aserción, respondemos **422 nombrando el ejercicio y el caso**, en lugar de descartar el campo en silencio.

Es a propósito. Falla cuando el docente publica el TP, que es barato, y no con un alumno esperando su corrección. Y descartar en silencio los dejaría creyendo que su contrato se respeta mientras les limpiamos el payload.

3.3 Un campo opcional más en la corrección, y por qué

Nuestro modelo exige que toda entrega pertenezca a una comisión, y ustedes no tienen comisiones. Su documento no podía verlo porque no ve nuestro modelo de datos.

Lo resolvemos con una **comisión de integración** que configuramos una vez por materia, más un campo **opcional** *comision_external_ref* en el cuerpo, por si alguna vez quieren modelar cohortes.

El contrato que ya implementaron no cambia. {alumno_ref, codigo, resultado_tests} sigue siendo válido tal cual.

3.4 Límite explícito de ejercicios por TP

Fijamos un tope de **50 ejercicios por trabajo práctico**. Su documento describe TPs de cuatro, así que hay margen de sobra. Lo decimos para que sea un límite conocido y no algo que descubran como un timeout. Si el uso real lo pide, lo subimos a propósito.

4. Dos correcciones a supuestos suyos

4.1 La clave del 409 de entregas (su §7.3)

Dicen que el 409 de *POST /entregas/* keyea por (*comision_id*, *rubrica_id*, *alumno_nombre*). El índice único real es (***rubrica_id***, ***alumno_nombre***), sin *comision_id*.

Su conclusión operativa es correcta y no cambia: comparar el *rubrica_id* en el match **no es opcional**. Pero como dijeron que los tres comportamientos de esa sección están codificados en su cliente, preferimos que el dato esté bien.

4.2 GET /entregas/{id} está arreglado

Lo reportaron al pasar y lo rodearon con `GET /correcciones/entregas/{id}`. Gracias por el dato: estaba roto **para todos**, no solo para ustedes.

La causa era que el servicio accedía al usuario que subió la entrega sin comprobar que existiera, y ese campo es nulo en toda entrega importada desde Moodle — que es por donde entra la mayoría de las entregas del sistema. Ya está corregido, con test de regresión. Cuando lo verifiquemos en staging les avisamos y pueden sacar el rodeo si quieren.

5. Una limitación que preferimos declarar

La anonimización cubrirá la base viva, no los respaldos históricos. Cuando implementemos el borrado por alumno (§3.6), el procedimiento va a destruir código, devolución y pseudónimo de la base en uso. Los respaldos de base de datos anteriores a esa operación quedan fuera de su alcance.

No lo escondemos porque afecta el alcance real del compromiso que ustedes firmaron con los alumnos del piloto. Si necesitan que los respaldos también queden cubiertos, hay que acordar una política de retención — es una decisión operativa, no de código, y es el punto 7 de nuestro pedido.

6. Lo que confirmamos que NO vamos a hacer

Tal cual su sección 5, para que no haya malentendidos:

- **No ejecutamos código.** El sandbox es de ustedes y funciona.
- **No calculamos la nota final del TP.** Devolvemos nota por ejercicio; el promedio ponderado lo hacen ustedes.
- **No escribimos nada de su lado.** La integración es de una sola dirección.
- **No tocamos el flujo de Moodle.** *cmid* sigue funcionando exactamente igual.
- **No hacemos corrección en lote.** Se dispara de a un ejercicio.

Cierre, y va en serio: el documento que mandaron está bien hecho. Que cada afirmación viniera con su fecha de verificación y su caso de evidencia nos ahorró días de discutir si los bugs existían y nos dejó ir directo a *por qué* existen. Los dos hallazgos del §2.2 salieron de poder leer el código buscando algo concreto en vez de a ciegas.

Quedamos a la espera de los puntos 1 a 3.